PROJECT CONCEPTS AND BUILD PLANS

ShellHacks 2026 project ideas

Ten concepts grounded in the winner case study

Choose a specific task, make its result visible, and build a small proof that the core works. Select the concept that matches your team and the confirmed event brief.

PLANNING BASIS	SCOPE
Historical evidence	31 overall winners and 5 supplementary category winners across 2017–2025
Current event	25–27 September 2026 • 36 hours • teams of 1–4
Purpose	Choose and scope a new project using traceable evidence and testable assumptions

Prepared by Codex for project planning. Independent analysis, not an official ShellHacks judging guide.
Companion to the winner case study; adapted from Smartsheet's existing Case Study Report Template.

What the data supports

Start with FormTrail if the team can obtain a clearly documented form and test its workflow. Choose Contract Canary for a team with backend and testing experience, or Counterexample Arcade for a small team comfortable with algorithms and animation. These are conditional feasibility recommendations, not predicted winning probabilities.

Across the primary cohort, all 31 projects have a specific intended task and 30 describe a workflow. Thirty also describe an interface or demonstrator. These observations favor an idea whose input, behavior and result can be explained clearly; they do not prove that a live demo worked or that these traits caused a win. [D01]

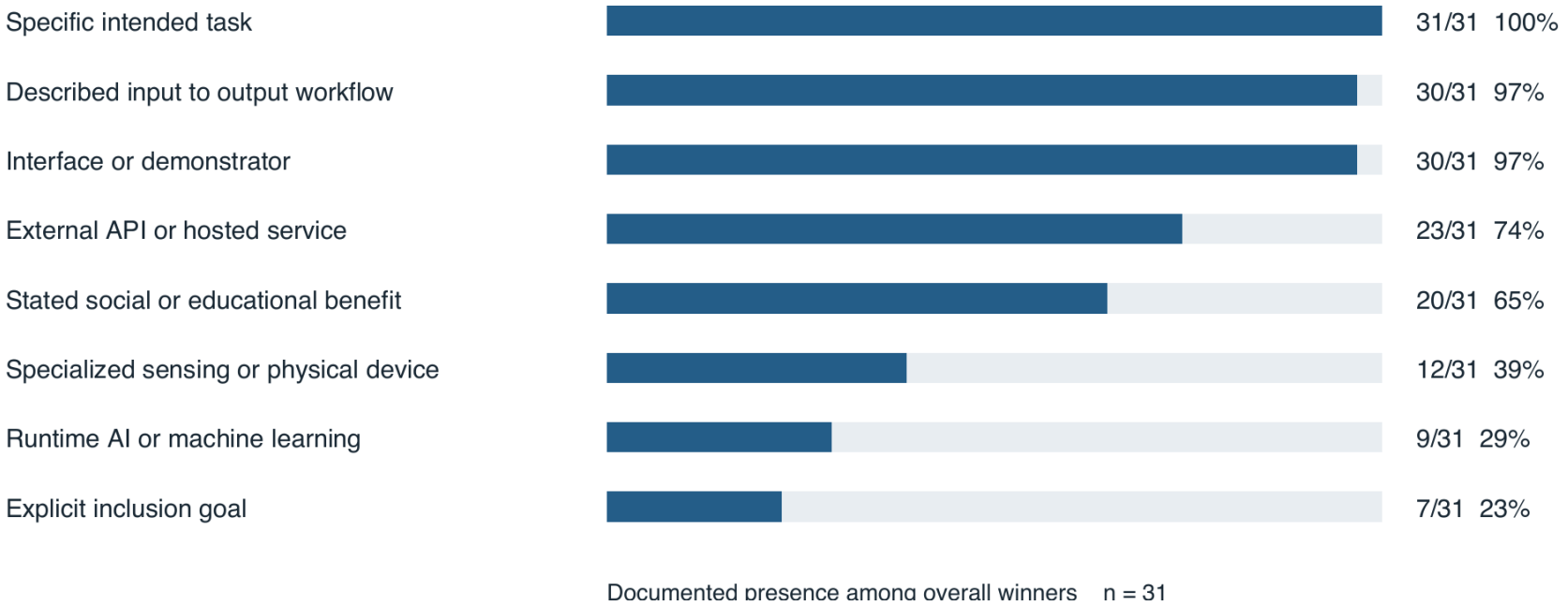


Figure 1. Documented traits among 31 overall podium projects. Categories overlap. An uncounted trait is not necessarily absent. Five supplementary category winners are excluded. Counts reproduced from the study's canonical dataset. [D01]

AI and hardware are options. Runtime AI or ML is established in 9/31 records and specialized sensing or devices in 12/31. A social or educational goal is stated in 20/31. These counts do not support a universal stack, an AI requirement or a mandatory social mission.

Every concept below is a proposal. Its user need, originality relative to existing products and effectiveness still require validation. Test counts and targets are future acceptance criteria, not collected results. The sample contains winners only, so no feature score or win probability is calculated.

Confirmed 2026 constraints

The public ShellHacks homepage and FAQ were rechecked on 25 September 2026. Build scope is based on the confirmed format. Prize-specific eligibility remains a separate check against the current hacker guide and challenge brief. [E26]

Confirmed public fact	Consequence for the project
25–27 September at FIU Graham Center, Miami; in person	Prepare a demonstration that can run at the venue and make its dependencies visible.
36-hour build window: Friday 11 p.m. to Sunday 11 a.m.	Deliver one complete interaction before adding breadth. Team estimates assume familiar tools and include time for rest, testing and the presentation.
Solo participation through teams of four	All concepts fit within that limit. Suggested team sizes are our estimates, not additional event requirements.
Students and graduates within one year from colleges, universities or coding bootcamps	Check each member’s eligibility. Concept suitability does not establish participant eligibility.

Not publicly verified at this cutoff: the 2026 judging rubric and weights, submission checklist, rules on prior work or AI tools, and individual sponsor challenge briefs. Public sponsor logos establish sponsorship; they do not establish a particular challenge. No concept here is asserted to qualify for an unverified prize.

Historical reference only. The 2025 rubric listed Completion, Originality, Design, Technology and Practicality. Those categories help structure a demonstration, but they are not presented as the 2026 rubric. The 2025 rules required event-created work and disclosure of reused code; confirm the current rules before using prior project code or preparing implementation. [E25, R25]

Before implementation, obtain the current brief, identify any required technology or deliverable, and compare two relevant existing tools with the chosen concept. Keep one sentence explaining the distinction you can actually demonstrate. The concepts differ from their historical precedents; global novelty has not been established.

Compare the ten concepts

The order highlights three practical starting points, then alternatives by interest. Effort and team estimates are planning judgments for the scoped prototype; they are not data-derived scores or a ranking of winning odds. Refer to each concept page for its evidence, assumptions and scope cuts.

Concept	Core proof	Team	Effort	Key dependency	Precedents
FormTrail	Find and resolve missing form requirements	2–3	Moderate	One documented form	P09, P24, P32
Contract Canary	Connect a response change to a broken interaction	2–3	Moderate	Instrumented test flow	P08, P18
Counterexample Arcade	Generate and repair a bounded counterexample	1–3	Lower	Correct finite test domain	P21, P23, P14
WindowWise	Explain whether an errand fits a class break	2–3	Lower	Verified service records	P24, P06
PantryRelay	Reserve the last item without overselling	3–4	Moderate	Transaction-safe updates	P20, P02
LostLink	Retrieve likely item matches and abstain	3–4	Higher	Model access and image set	P13, P29
LineCheck	Trace a mismatch to document source lines	3–4	Higher	OCR with source locations	P33, P13
Pulse Path	Decode a tactile cue and replay the route	2–4	Higher	Hardware and embedded skills	P27, P17
StormTable	Replay resource choices using explicit rules	3–4	Moderate	Transparent simulation rules	P04, P11, P30
BranchBridge	Prove that joint choices change the ending	2–4	Moderate	Shared game-state logic	P23, P36, P12

Table 1. All team estimates fall within the confirmed 1–4 person limit. Lower effort means fewer external dependencies within this shortlist; it does not mean trivial implementation. P32, P33 and P36 are supplementary category winners, not primary-cohort observations.

If user access is limited, Counterexample Arcade and Contract Canary allow deterministic testing without institutional integration. If hardware or unfamiliar model access is uncertain, resolve that dependency early before choosing Pulse Path, LostLink or LineCheck.

I01 FormTrail

Guide a student through one complex campus form, linking every requirement to its source and showing exactly what remains incomplete.

Accessible campus workflow • Suggested team 2–3 • Effort Moderate • AI not needed for the core

Core interaction. Open supported form → see source-linked checklist → jump to a field → save draft → review missing items → export a review copy.

Why the data supports this direction. No Vote Left Behind connected an access problem to form processing; IntelliSchedule placed useful information inside an existing student task. A11y Tools, a supplementary category winner, adapted web interactions. Together they support a focused in-context workflow, not a claim that accessibility increases winning odds. [P09, P24, P32]

Distinctive feature. The central feature is traceable completeness: distinguish a blank field, a missing attachment and an answer still needing review. Support one known form with manually mapped requirements; the product is not a universal form-filling agent.

36-hour prototype. One permission-compatible form or clearly labeled local fixture, keyboard-accessible checklist, field highlighting, local draft storage and export. Use ordinary browser DOM mapping and deterministic rules. Two people can own the interface and rules; a third can test accessibility and the pitch.

Judge-facing demonstration. Give a judge a deliberately incomplete form. Jump from two checklist items to the original instructions and fields; correct them; show the review state update and export. Do not submit to a real institution.

Proposed validation. Create ten seeded omissions and five valid examples; target all ten detected with zero false flags. Test the entire journey by keyboard. With five volunteers, counterbalance guided and original versions and report task time, omissions and the small sample size.

Scope and dependency. No automatic submission, eligibility decisions, universal website support or generated institutional requirements. Add language simplification only after the core works and retain the original instructions beside it. A real, clearly documented form is the strongest input. If unavailable by hour 3, use a labeled fixture and describe user-need validation as pending.

Conditional challenge fit. Consider an accessibility or student-experience prize only if its actual 2026 brief includes this use case. No current prize eligibility is established.

I02 Contract Canary

Show which user interaction breaks after an API response changes, with a replayable failing example and a visual explanation.

Developer tool • Suggested team 2–3 • Effort Moderate • AI not needed for the core

Core interaction. Load two API fixtures → replay one instrumented user flow → compare outcomes → identify changed fields → open a minimal failing interaction.

Why the data supports this direction. Freetear won first overall with a narrowly defined developer setup workflow. Pegasus, first in the virtual division, converted developer inputs into inspectable findings. The transferable pattern is removing a concrete development obstacle and making the result visible. [P08, P18]

Distinctive feature. Move beyond a raw JSON diff to a before-and-after application interaction. The prototype explains a failure in one instrumented flow; it does not promise to infer arbitrary application behavior.

36-hour prototype. A local checkout demo, two API versions, JSON fixture loader, explicit field-to-component mapping, replay runner and side-by-side trace. One teammate builds the runner, one the interface and a third the labeled examples. No cloud service is necessary.

Judge-facing demonstration. Rename an amount field or change its type. Show the checkout fail, open the response difference and replay the corrected contract. Also show an extra harmless field producing no break alert.

Proposed validation. Prepare twelve labeled mutations, balanced between breaking and harmless changes. Report the confusion matrix, false alarms and replay repeatability. Keep some mutations unseen until the demo rehearsal; do not report fixture performance as general compatibility accuracy.

Scope and dependency. One endpoint family and one application flow. Exclude production traffic, arbitrary authentication, automatic fixes and universal causal diagnosis. A teammate comfortable with request handling and a small test harness. If interaction tracing is unfinished by hour 10, use explicit field dependencies and show that limitation.

Conditional challenge fit. Potential developer-tool or technical-execution fit if such a 2026 brief exists. A sponsor integration should perform a necessary function, not merely host a logo.

I03 Counterexample Arcade

Teach binary search by letting learners break its rules, discover a small failing input and repair the mistake through animation.

Learning game • Suggested team 1–3 • Effort Lower • AI not needed for the core

Core interaction. Change a predefined rule → search a bounded input set → animate a counterexample → explain the failure → repair the rule → replay.

Why the data supports this direction. AlgoSaurus made algorithms visible and won third in person; Chess 10 won overall with a changed-rule game. Tango connected practice with visual feedback. These precedents support an interactive learning loop with an observable consequence, not proof of learning effectiveness. [P21, P23, P14]

Distinctive feature. The learner creates and fixes a failure. The distinctive demonstration is the smallest failing case within the tested bounds, rather than another animation of a correct sorting algorithm.

36-hour prototype. Binary search only, three editable rule choices, sorted arrays of length 0–6 over {0,1,2,3}, targets in {-1,0,1,2,3,4}, an iteration cap and animated pointers. Compare target presence with a reference implementation; any correct matching index is valid when values repeat.

Judge-facing demonstration. Ask the judge to change the loop boundary. Generate a small failing array, step through the missed target, repair the rule and rerun the same input successfully.

Proposed validation. Enumerate all 1,260 array/target pairs in this finite domain, ordered by length, array values and target. Verify counterexample minimality only within that ordering and domain. Ask five learners to explain the bug before and after; report observations without claiming a controlled learning study.

Scope and dependency. No arbitrary code execution, universal program verification, multiple programming languages or broad algorithm catalog. Describe bounded testing precisely. Accurate algorithm logic and clear animation matter more than external APIs. If counterexample minimization is late, enumerate inputs in a documented size order.

Conditional challenge fit. Potential education or interactive-experience fit if confirmed in 2026. Runtime AI adds little to the core proof; use it only for a separately evaluated feature.

I04 WindowWise

Find a feasible time for one campus errand between classes, accounting for office hours, walking time and required documents.

Campus planning • Suggested team 2–3 • Effort Lower • AI not needed for the core

Core interaction. Enter a class break and errand → check source-dated service rules → calculate a feasible visit → explain a conflict → revise the plan.

Why the data supports this direction. IntelliSchedule combined outside information for a student decision and won second overall. JAM organized scattered application information and received a third-place award. The reusable pattern is consolidation followed by a specific next action. [P24, P06]

Distinctive feature. Solve a constrained feasibility question: can this particular visit fit? A clear rejection with the conflicting time or missing document is part of the product, not an error screen.

36-hour prototype. Six service records, manually entered timetable, editable walking-time matrix, prerequisite checklist and source/verification date for each record. A browser app with deterministic interval checks is enough; no student-portal integration is required.

Judge-facing demonstration. Start with a feasible visit. Shorten the break, move the next class or mark a required document missing. Show the recommendation disappear and the exact constraint responsible.

Proposed validation. Use twenty hand-calculated cases covering exact closing times, travel buffers and prerequisites. Compare five users completing the same planning task against separate source pages; counterbalance order and report assumptions about walking and service time.

Scope and dependency. One campus and one errand at a time. Exclude live queue forecasts, institutional login, enrollment actions and dependent scraping. Use manually verified public hours when possible. Label invented demonstration records as fixtures; do not imply that assumed service duration guarantees a completed visit.

Conditional challenge fit. Potential student productivity or community brief fit if verified. The prototype can remain useful without adding an AI scheduler.

I05 PantryRelay

Reserve available pantry supplies for a pickup window and ensure that the last item cannot be promised to two people.

Community resource workflow • Suggested team 3–4 • Effort Moderate • AI not needed for the core

Core interaction. View available quantity → reserve a pickup window → hold inventory → scan once at collection → release an expired reservation.

Why the data supports this direction. DAOchow, second virtual, addressed shared community food resources. Pocket Park, second overall, connected available inventory with reservations. Their combination suggests an inspectable resource-to-handoff workflow; it does not establish demand at a particular pantry. [P20, P02]

Distinctive feature. Make reliable handoff the technical centerpiece: simultaneous claims, expiration and duplicate pickup handling. The scope avoids recreating DAO funding or a broad marketplace.

36-hour prototype. Twelve synthetic inventory items, two student sessions, one volunteer dashboard, transactional reservation logic and a single-use pickup token. Assign backend consistency, student UI, volunteer UI and testing across the team.

Judge-facing demonstration. Two browsers request the last item at the same time. Exactly one reservation succeeds. Collect it, repeat the scan and show that inventory does not decrement twice; then show a different reservation expire.

Proposed validation. Run at least twenty paired concurrent requests, ten expiry cases and ten repeated scans. Target zero overselling and duplicate collection in these tests. Ask a resource-desk volunteer to review the workflow if available; otherwise mark the operational assumptions as unvalidated.

Scope and dependency. One location; no payments, delivery, eligibility determinations or nutrition advice. Use demonstration inventory until a real operator authorizes integration. The team needs someone able to implement transaction-safe inventory updates. If that is unfamiliar, select a simpler concept rather than presenting a race-prone reservation system as complete.

Conditional challenge fit. Potential community or social-benefit fit under an actual 2026 brief. A blockchain or agent layer is not needed to demonstrate the core.

I06 LostLink

Match a lost-item description or photograph to a small found-item catalog, explaining candidate matches and allowing no match.

Visual search • Suggested team 3–4 • Effort Higher • AI useful for visual retrieval

Core interaction. Submit description or image → retrieve candidate items → show matching and conflicting attributes → shortlist for human review.

Why the data supports this direction. CloudPlate, third overall, connected camera recognition to a record and an action. EcoSort, a 2025 overall winner, connected an image to a practical next step. Transfer that input-to-record-to-action pattern. EcoSort is not evidence that runtime AI was implemented; its coding platform alone does not establish that. [P13, P29]

Distinctive feature. Focus on ambiguous matches and useful abstention: two similar bags should remain uncertain. Explain which visible attributes agree instead of asserting ownership from a model score.

36-hour prototype. A consented or staged catalog of thirty objects, one visual embedding or attribute-extraction component, text filters, candidate view and no-match state. Keep retrieval separate from custody or item release.

Judge-facing demonstration. Search for a distinctive bottle and inspect its candidates. Then show two similar black backpacks and an absent item; explain the uncertainty and abstention behavior.

Proposed validation. Create twenty alternate-photo queries, including at least five absent objects. Report top-three retrieval success for present items and false matches on absent items. Compare with a keyword/color baseline; keep evaluation images separate from catalog images and do not tune on the final test queries.

Scope and dependency. No ownership verification, facial recognition, campus-wide rollout or automated release of property. Text search is the honest fallback if visual retrieval adds no measured value. Model access, latency and a useful image set are early gates. Check them by hour 3; unfamiliar model training is outside the 36-hour plan.

Conditional challenge fit. Consider an actual vision or AI prize only if the model materially improves the measured retrieval task and its required technology is supported.

I07 LineCheck

Compare a purchase order with an invoice, highlighting quantity, price and total discrepancies with links to the source locations.

Document reconciliation • Suggested team 3–4 • Effort Higher • AI useful for ocr and extraction

Core interaction. Upload two synthetic documents → extract supported fields → confirm uncertain text → reconcile line items → highlight each discrepancy in context.

Why the data supports this direction. AgentZero, a supplementary Capital One category winner, described invoice analysis. CloudPlate connected OCR with structured records. The transferable pattern is turning an unstructured document into an inspectable action; neither source establishes validated detection accuracy. [P33, P13]

Distinctive feature. Provide a verifiable source trail and arithmetic reconciliation. Show exactly why a line differs; do not claim to infer fraud or financial risk from a mismatch.

36-hour prototype. Two fixed document layouts, a preexisting OCR or document model, bounding-box references, a review screen and deterministic arithmetic. Support one currency and explicit line identifiers. Separate extraction, matching and presentation responsibilities.

Judge-facing demonstration. Upload an order and invoice with one changed quantity and one OCR ambiguity. Correct the ambiguous extraction; show that one alert clears while the real discrepancy remains tied to its source line.

Proposed validation. Use ten document pairs with twenty deliberately seeded discrepancies and clean controls. Report extraction errors separately from reconciliation precision and recall. Include at least one held-out layout variation within the declared supported family; disclose any unsupported cases.

Scope and dependency. No payments, tax interpretation, fraud labels, real customer documents or arbitrary accounting integrations. Use synthetic examples and a manual field-entry fallback. Verify OCR access and bounding-box support by hour 3. If source locations cannot be retained, narrow the input to a supported format instead of inventing evidence links.

Conditional challenge fit. Potential finance or document-AI fit only if the 2026 brief is compatible. The 2025 Capital One prize does not establish a 2026 prize.

I08 Pulse Path

Teach four tactile direction cues through a tabletop maze, then replay the player's route and mistakes on a companion screen.

Hardware learning game • Suggested team 2–4 • Effort Higher • AI not needed for the core

Core interaction. Choose route → feel a cue → press a direction button → record response → reveal path and errors → retry.

Why the data supports this direction. ARSL, second overall, combined learning with physical feedback. The Tilter, first in person, made a sensor-controller-output loop directly observable. The transfer is a tangible interaction whose inputs and results can be inspected immediately. [P27, P17]

Distinctive feature. A bounded tactile-language learning game replaces a broad navigation or recognition system. The companion replay makes the physical interaction explainable even to judges who cannot feel the device.

36-hour prototype. One microcontroller, available vibration hardware, four button inputs, four documented cue patterns and one maze. One teammate handles electronics, another the game and a third the replay interface. Use proper low-voltage motor-driving components already available to the team.

Judge-facing demonstration. Teach the four cues, let a judge try a short route, then reveal each cue and response on the screen. Repeat the same route and compare errors without claiming a validated learning effect.

Proposed validation. Bench-test fifty cue/response cycles for correct output and event logging. Record response latency, wrong turns and completion time for a few volunteers. Treat repeated-attempt improvements as exploratory observations.

Scope and dependency. No real-world navigation, autonomous movement, custom PCB or claims of accessibility effectiveness. A visual simulator is a separate fallback prototype and must be labeled as such. Choose this only with hardware in hand and embedded experience. If a reliable cue cannot be demonstrated by hour 4, switch to the simulator or another concept.

Conditional challenge fit. Potential hardware or learning brief fit if present in 2026. Historical hardware wins show precedent, not a hardware bonus.

I09 StormTable

Run a fictional neighborhood preparedness exercise in which stale reports and limited resources change the outcome of each decision.

Preparedness simulation • Suggested team 3–4 • Effort Moderate • AI not needed for the core

Core interaction. Load fictional map → inspect timestamped reports → allocate a limited resource → reveal a scripted event → compare decisions and unmet requests.

Why the data supports this direction. HurriHome combined home status and a neighborhood view; Farsense connected remote sensors to a dashboard; FloodGuard, a 2025 overall winner, described a multi-source flood workflow. These precedents support making distributed information understandable. They do not validate disaster predictions or response outcomes. [P04, P11, P30]

Distinctive feature. Teach information freshness and constrained decisions through a transparent simulation. Every outcome comes from visible rules and fictional inputs, rather than a claimed real-time forecast.

36-hour prototype. One fictional district, six locations, three scripted events, two resource types, timestamps and a deterministic rules engine. Build the state model first, then the map and debrief view. Browser graphics are sufficient.

Judge-facing demonstration. Give the judge two reports with different ages. Allocate a resource, reveal a changed condition and replay an alternative decision. Show how the declared simulation rules produced the different result.

Proposed validation. Test resource conservation, deterministic replay and expired-report handling across twelve scenarios. Ask five players to identify stale evidence and explain a tradeoff; report this as usability feedback, not real-world preparedness improvement.

Scope and dependency. No real alerts, evacuation routes, live hazard predictions, municipal integration or claims that a simulated optimum is safe in reality. Keep the simulation's rules explicit and small. If the team cannot explain an outcome from the rules, remove that mechanic instead of generating a plausible explanation afterward.

Conditional challenge fit. Potential education, community or simulation fit under a compatible 2026 brief. Agents are optional only where independent subtasks and their outputs are genuinely observable.

I10 BranchBridge

Give two players different evidence in a short mystery; their joint choices change the ending and a replay shows which decisions mattered.

Cooperative narrative game • Suggested team 2–4 • Effort Moderate • AI not needed for the core

Core interaction. Join two roles → inspect different clues → communicate → make a joint choice → reach an ending → inspect the decision trail.

Why the data supports this direction. Chess 10 won overall with altered game rules. Quests and Conflicts VR, a supplementary Netflix category winner, used branching interactive entertainment. Social Distance Beer Pong won second overall with an inspectable physical game interaction. These cases establish entertainment as a legitimate historical direction. [P23, P36, P12]

Distinctive feature. Asymmetric evidence changes what each player knows, and the final replay proves that choices affected state. The core is cooperation and consequence, independent of VR or generated dialogue.

36-hour prototype. Two browser roles, one shared state machine, three consequential choices, two endings and reconnect support. Use a short authored script and simple art. Two teammates can split synchronization and the narrative/interface.

Judge-facing demonstration. Let judges take the two roles. Reveal that each has a different clue; record a joint choice; finish the short scene; then replay an alternative branch and show the changed ending.

Proposed validation. Enumerate every reachable branch and verify ending consistency. Test duplicate actions and reconnection during ten paired sessions. Ask players to explain which choice changed the outcome; report observed understanding rather than assumed engagement.

Scope and dependency. One short scene; no open world, generated story engine, headset requirement or large art pipeline. Keep the session short enough to demonstrate within the actual judging slot. Synchronization must preserve one shared decision state. If networking is unreliable, a labeled local two-role mode can preserve the core mechanic.

Conditional challenge fit. Potential interactive entertainment fit only if offered in 2026. Netflix's 2025 branching-narrative challenge is historical, not a current eligibility claim.

Turn the concept into a convincing entry

Use the actual 2026 judging and submission instructions when available. The following proof checklist adapts the published 2025 categories for preparation; it carries no official 2026 weights. Each item should be backed by something a judge can inspect. [E25]

Historical category	Prepare this evidence
Completion	One end-to-end interaction, visible error handling and an explicit list of unfinished features.
Originality	A comparison with two relevant existing products and one distinctive behavior demonstrated in the prototype.
Design	A clear primary action, legible feedback and an interface tested with the input methods the target user needs.
Technology	A small architecture diagram or trace showing which component performs the difficult work. Label simulations, fixtures and model calls.
Practicality	A timed task or correctness test, the number and kind of examples tested, and observed failures. Do not turn small tests into broad impact claims.

A suggested 36-hour sequence

Elapsed hours	Outcome
0–3	Confirm current rules and challenge fit; validate the proposed need; check the riskiest dependency.
3–10	Build a thin end-to-end version. If the central interaction is absent, cut scope immediately.
10–22	Complete the core behavior and error states. Stagger rest and integrate frequently.
22–28	Run the declared tests; retain failures and measured results. Conduct a small usability check if feasible.
28–33	Improve clarity, rehearse within the real judging slot, record a labeled backup and prepare the submission.
33–36	Freeze optional features, verify required materials and keep a submission buffer.

Recommendation. Select the smallest concept that demonstrates a meaningful distinction and that your team can test honestly. The historical data supports a broad range of technologies; the defensible common thread is a specific task with an inspectable result. Current rules and the team’s actual execution should determine the final choice.

Evidence register

Project IDs match the original case study and canonical dataset. Award and capability statements derive from participant-maintained Devpost pages; prototype behavior and impact were not independently tested. A concept borrows a pattern, not evidence that its proposed implementation will work.

P02 Pocket Park | 2017 Project source

Overall podium cohort. Hosts list spaces and prices; drivers choose a map location and reserve parking.

P04 HurriHome | 2018 Project source

Overall podium cohort. Home sensors send status to a cloud platform, mobile app and neighborhood map.

P06 JAM | 2018 Project source

Overall podium cohort. Dashboard compares application stage, location and compensation.

P08 Freetear | 2019 Project source

Overall podium cohort. CLI templates an application and automates connections to free-tier deployment services.

P09 No Vote Left Behind | 2019 Project source

Overall podium cohort. Raspberry Pi generates QR/RFID credentials; a web form fills an official registration PDF and sends it to the relevant jurisdiction.

P11 Farsense | 2020 Project source

Overall podium cohort. Battery-powered sensor stations send weather observations over a LoRa mesh to a dashboard.

P12 Social Distance Beer Pong | 2020 Project source

Overall podium cohort. Infrared cup sensors and a microcontroller send data by USB to a C++ terminal game.

P13 CloudPlate | 2020 Project source

Overall podium cohort. Camera/OCR identifies an arriving car; dashboard displays customer and service records; personalized SMS greets the customer.

P14 Tango | 2021 Project source

Overall podium cohort. Pose estimation compares a learner to a reference routine; timing and body-size adjustments support live annotations, charts and replay.

P17 The Tilter | 2022 Project source

Overall podium cohort. Phone camera tracks the ball; Python PID control sends serial commands to Arduino servos.

P18 Pegasus | 2022 Project source

Overall podium cohort. Upload CSV data for indexed queries or source code for rule-based vulnerability scanning and alerts.

P20 DAOchow | 2022 Project source

Overall podium cohort. A decentralized web app connects users to food access and manages funds through a DAO.

Evidence register continued

P21 AlgoSaurus | 2022 Project source

Overall podium cohort. Choose an algorithm, observe animated sorting, and adjust chart size or speed while reading explanations.

P23 Chess 10 | 2023 Project source

Overall podium cohort. Play a 3D chess variant extending en passant mechanics to most pieces.

P24 IntelliSchedule | 2023 Project source

Overall podium cohort. Chrome extension places ratings, difficulty, return-interest scores and review summaries alongside FIU course choices.

P27 ARSL | 2024 Project source

Overall podium cohort. Quest passthrough identifies objects; hand tracking and a vibrating glove guide the learner through spelling.

P29 EcoSort | 2025 Project source

Overall podium cohort. Photograph an item, receive disposal guidance, review scans and find donation or drop-off locations.

P30 FloodGuard | 2025 Project source

Overall podium cohort. Parallel agents monitor weather, infrastructure and alerts, then coordinate risk notifications.

P32 A11y Tools | 2025 Project source

Supplementary category winner. Choose extension tools for voice interaction, image description, text simplification, search or font changes.

P33 Agent Zero | 2025 Project source

Supplementary category winner. Route invoice checks to specialized agents and aggregate risk scores into a readable assessment.

P36 Quests and Conflicts VR | 2025 Project source

Supplementary category winner. Interact with a VR narrative whose paths respond to player choices.

Event and study sources

E26 Official ShellHacks 2026 homepage and FAQ — Current format and eligibility; rechecked 25 September 2026.

E25 ShellHacks 2025 event overview — Historical judging categories and sponsor briefs; not 2026 requirements.

R25 ShellHacks 2025 rules — Historical build and reuse rules; current rules take priority.

T01 Smartsheet Case Study Report Template — Existing template adapted consistently with the companion study. Smartsheet does not author or endorse this analysis.

D01 ShellHacks winner case study and dataset, version 1.0.0. Primary cohort: 31 overall projects across all nine completed editions, 2017–2025. Five supplementary 2025 category winners are analyzed separately. Companion files: ShellHacks_Winners_Case_Study.pdf, ShellHacks_Winners_Dataset.xlsx and dataset.json.

Reproducible concept records: project_ideas_2026.json. All measurements described for the new concepts are proposed validation plans. No private accounts, participant contact details or nonpublic judging materials were used.